

Rider POV Object Detection and Eye Gaze Correlation Using YOLO-Based Models

A Sai Teja

Department of Computer Science and Engineering, IIITS

Submitted: March 31, 2026

Abstract—This report presents a methodology for detecting road objects from a rider’s point-of-view (POV) camera and correlating the detected scene with eye tracking gaze data. A dashcam video was recorded from a motorcycle rider’s perspective. Frames were extracted at 1 FPS using a custom Python script and uploaded to Roboflow for manual annotation across three classes: *Car*, *Pedestrian*, and *Intersection Conflict Zone*. Five model versions (Ver 0–4) were trained using different backbone architectures—Roboflow RF-DETR (Nano), YOLOv11 (Fast), YOLOv12 (Fast), and YOLOv26 (Fast)—with progressive dataset augmentation strategies. The best-performing model (Ver 2, RF-DETR Nano) achieved a validation mAP@50 of 57.3%, a precision of 64.3%, and a recall of 43.3%. This report documents the complete experimental pipeline including dataset construction, preprocessing, model training, and evaluation across all versions.

Keywords—Object Detection, YOLO, RF-DETR, Eye Tracking, Rider POV, Roboflow, mAP

1. INTRODUCTION

Traffic accidents involving motorcyclists and cyclists remain a significant road safety challenge, particularly at intersections and conflict zones where multiple road users interact. Understanding where a rider directs their visual attention—and whether they are looking at safety-critical objects—can provide valuable insights for developing intelligent driver assistance systems.

This project aims to build a pipeline that: (1) detects road objects from a rider’s POV video using YOLO-based detectors, and (2) correlates the detected scene with eye-tracking gaze data to determine whether the rider was fixating on relevant objects such as pedestrians, cars, or intersection conflict zones.

The core challenge is that rider POV footage is highly dynamic—camera motion, lighting variation, and occlusion make annotation and detection difficult. This report documents all experimental steps, from raw video processing to model evaluation across five training versions.

2. DATASET CONSTRUCTION

2.1. Video Recording

The source material is a single dashcam video (scene_with_gaze.mp4) captured from a motorcycle rider’s handlebar-mounted camera. The video contains typical urban riding scenarios including vehicles, pedestrians, and intersection zones.

2.2. Frame Extraction at 1 FPS

To generate annotatable frames, a Python script (video_frame_extractor.py) was written to sample one frame per second from the source video. This rate was chosen

to capture scene changes while avoiding redundancy between consecutive frames.

```
import cv2, os
from pathlib import Path

def extract_frames_at_1fps(video_path,
                           output_dir="extracted_frames"):
    Path(output_dir).mkdir(parents=True,
                           exist_ok=True)
    cap = cv2.VideoCapture(video_path)
    fps = cap.get(cv2.CAP_PROP_FPS)
    frame_interval = int(fps)
    frame_count, saved_count = 0, 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        if frame_count % frame_interval == 0:
            timestamp = frame_count / fps
            fname = (f"frame_{saved_count:04d}"
                    f"_t{timestamp:.2f}s.jpg")
            cv2.imwrite(
                os.path.join(output_dir, fname),
                frame)
            saved_count += 1
            frame_count += 1
        frame_count += 1
    cap.release()
```

Code 1. Frame extraction at 1 FPS

The output frames were saved to the extracted_frames/ directory, named with zero-padded indices and timestamps (e.g., frame_0042_t42.00s.jpg) for traceability.

2.3. Manual Annotation on Roboflow

Extracted frames were uploaded to the Roboflow annotation platform. The following object classes were defined:

- Auto
- Bill Board
- Bus Heavy Vehicle
- Car
- Intersection Conflict Zone
- Pedestrian
- Sign Boards
- Traffic Signal Board
- Truck Heavy Vehicle
- Two Wheeler

COLOR	CLASS NAME	COUNT
	Auto	474
	Bill Board	9
	Bus Heavy Vehicle	98
	Car	1,293
	Intersection Conflict Zone	56
	Pedestrian	742
	Sign Boards	66
	Traffic Signal Board	8
	Truck Heavy Vechile	93
	Two Wheeler	4,428

Figure 1. Object classes used for annotation in the final dataset.

Bounding boxes were drawn manually for each class in every frame. The total dataset comprises approximately **875 base images** before augmentation.

3. PREPROCESSING AND AUGMENTATION

A consistent base preprocessing pipeline was applied to all model versions:

- **Auto-Orient** — corrects EXIF rotation metadata
- **Auto-Adjust Contrast** — normalizes image contrast

For Versions 3 and 4, an additional **Crop** augmentation was introduced, which effectively doubled the dataset size by producing zoomed-in views of each image. This brought the training set for those versions to approximately 1,750 images.

The augmentation pipeline applied to the training split included:

- Horizontal Flip
- Random Noise (up to 0.3%)
- 90° Rotation (clockwise and counter-clockwise)

Table 1 summarises the dataset configuration across versions.

Table 1. Dataset size and preprocessing per version

Version	Base Images	With Crop
Ver 0	875	No
Ver 1	875	No
Ver 2	875	No
Ver 3	875	Yes (×2)
Ver 4	875	Yes (×2)

4. MODEL VERSIONS AND TRAINING

Five model versions were trained and evaluated. Each version used a different model architecture or dataset configuration. All models were trained using Roboflow’s managed training infrastructure.

4.1. Version 0 — RF-DETR Nano (Baseline)

INFO

Ver 0 served as the initial proof-of-concept. It used the Roboflow RF-DETR Object Detection (Nano) model evaluated on the Roboflow Instant 2 dataset split (ID: annotating-images-pg1js/3).

Validation Metrics:

- mAP@50: 38.0%
- Precision: 33.3%
- Recall: 33.3%

The training graphs for Ver 0 are shown in **Fig. 2** and **Fig. 3**. The mAP curve is noisy, peaking around epoch 3 (37%) before declining to approximately 20% by epoch 18. The loss curves (Box Location, Class, Box Overlap) all trend upward over training, suggesting overfitting or insufficient data diversity at this stage.

Per-class mAP@50 results on the validation set (**Fig. 4**) reveal a severe imbalance: *Car* achieves 100% mAP@50 as the dominant class, while *Intersection Conflict Zone* achieves only 1% and *Pedestrian* achieves 13%.

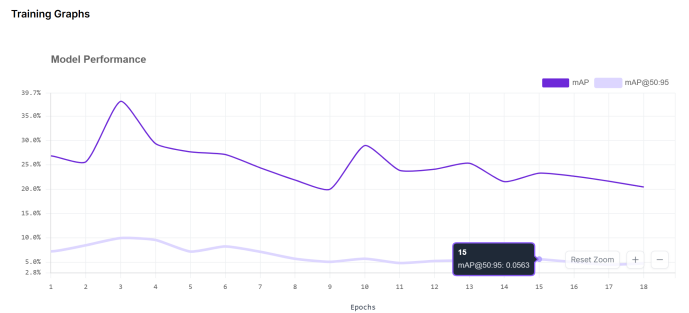


Figure 2. Ver 0 — Model performance (mAP and mAP@50:95) over 18 epochs.



Figure 3. Ver 0 — Box location loss, class loss, and box overlap loss over training.



Figure 4. Ver 0 — Average Precision by class (mAP@50) on the validation set.

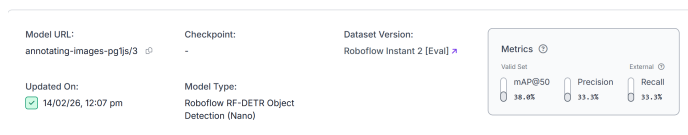


Figure 5. Ver 0 — Model details: RF-DETR Nano, trained on Roboflow Instant 2.

4.2. Version 1 — YOLO26 (Fast)

INFO

Ver 1 used the YOLOv26 Object Detection (Fast) architecture. Dataset: *Final Version* (ID: annotating-images-pg1js/13).

Validation Metrics:

- mAP@50: 33.8%
- Precision: 91.6%
- Recall: 17.0%

Ver 1 exhibited an extreme trade-off: very high precision (91.6%) but very low recall (17.0%). The model was highly conservative—it made very few predictions, and those it did make were mostly correct, but it missed the vast majority of true positive instances. The low mAP@50 (33.8%) reflects this imbalance.

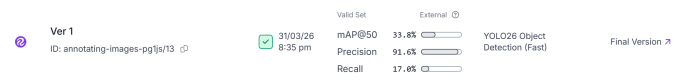


Figure 6. Ver 1 — Summary metrics from Roboflow training output.

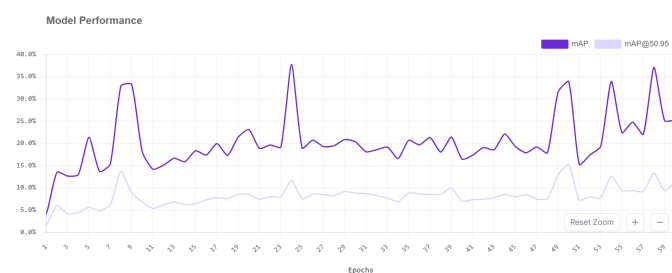


Figure 7. Ver 1 — Model performance (mAP and mAP@50:95) over training epochs.



Figure 8. Ver 1 — Training loss curves (box, class, and overlap components).



Figure 9. Ver 1 — Per-class validation precision (mAP@50).



Figure 10. Ver 1 — Model and dataset training configuration details.

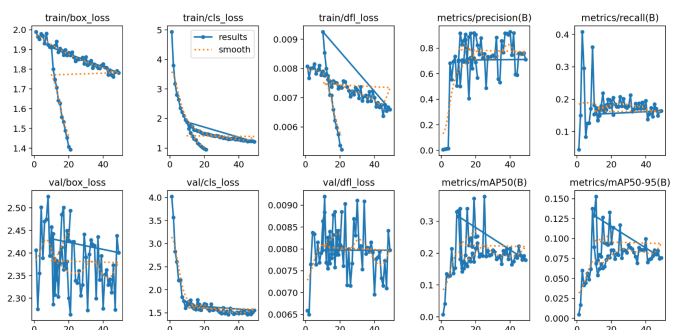


Figure 11. Ver 1 — Advanced training graphs exported from Roboflow.

4.3. Version 2 — RF-DETR Nano (Final Dataset)

INFO

Ver 2 used the same RF-DETR Nano architecture as Ver 0, but trained on the improved *Final Version Data Set* (ID: annotating-images-pg1js/14).

Validation Metrics:

- mAP@50: 57.3%
- Precision: 64.3%
- Recall: 43.3%

Ver 2 is the **best-performing model** in this study. The improvement over Ver 0 (from 38% to 57.3% mAP@50) can be attributed primarily to dataset refinement—better annotations and a cleaner training split. Both precision and recall improved substantially compared to the baseline.

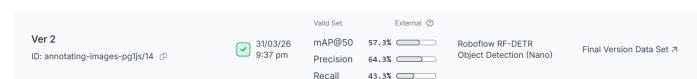


Figure 12. Ver 2 — Summary metrics from Roboflow training output.

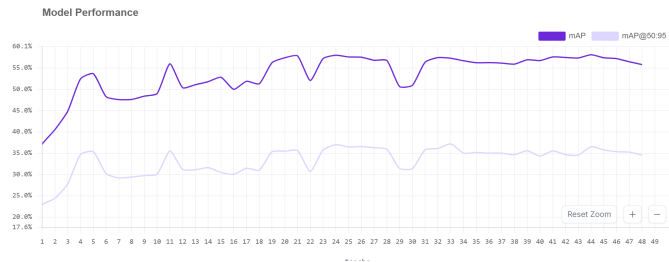


Figure 13. Ver 2 — Model performance (mAP and mAP@50:95) over training epochs.



Figure 14. Ver 2 — Training loss curves (box, class, and overlap components).



Figure 15. Ver 2 — Per-class validation precision (mAP@50).



Figure 16. Ver 2 — Model and dataset training configuration details.

4.4. Version 3 — YOLOv11 (Fast) with Crop

INFO

Ver 3 introduced the crop augmentation, doubling the dataset. Architecture: YOLOv11 Object Detection (Fast) (ID: annotating-images-pg1js/16).

Validation Metrics:

- mAP@50: 46.9%
- Precision: 69.2%
- Recall: 42.5%

Ver 3 achieved a balanced precision-recall trade-off. The crop augmentation helped improve object-scale diversity, and YOLOv11 Fast is a lighter, faster architecture suited for real-time deployment. Though the mAP@50 (46.9%) is below Ver 2, the precision (69.2%) is the highest outside of the degenerate Ver 1 result.

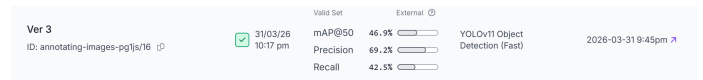


Figure 17. Ver 3 — Summary metrics from Roboflow training output.

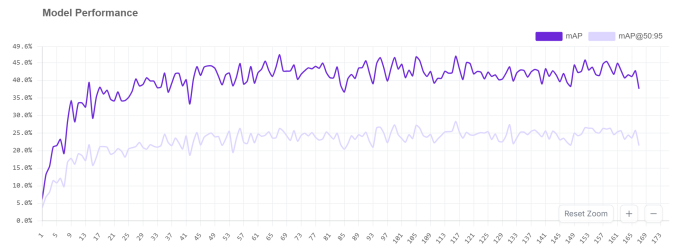


Figure 18. Ver 3 — Model performance (mAP and mAP@50:95) over training epochs.

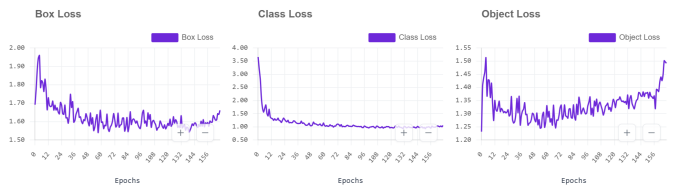


Figure 19. Ver 3 — Training loss curves (box, class, and overlap components).



Figure 20. Ver 3 — Per-class validation precision (mAP@50).



Figure 21. Ver 3 — Model and dataset training configuration details.

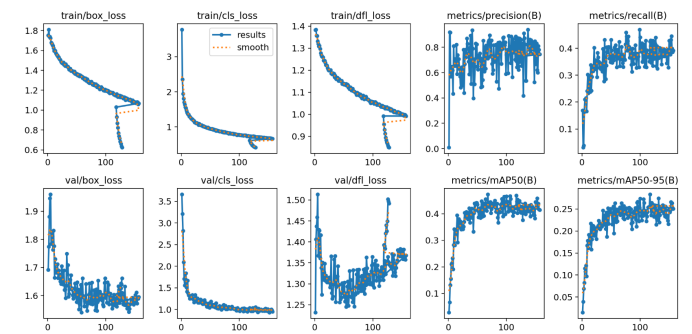


Figure 22. Ver 3 — Advanced training graphs exported from Roboflow.

4.5. Version 4 — YOLOv12 (Fast) with Crop

INFO

Ver 4 used the newer YOLOv12 Object Detection (Fast) architecture with the same crop-augmented dataset as Ver 3 (ID: annotating-images-pg1js/17).

Validation Metrics:

- mAP@50: 46.0%
- Precision: 77.1%
- Recall: 40.2%

Ver 4 completed training with a strong precision-focused profile. Compared with Ver 3, it delivers higher precision but slightly lower recall, while remaining in a similar mAP@50 range.



Figure 23. Ver 4 — Summary training snapshot from Roboflow.

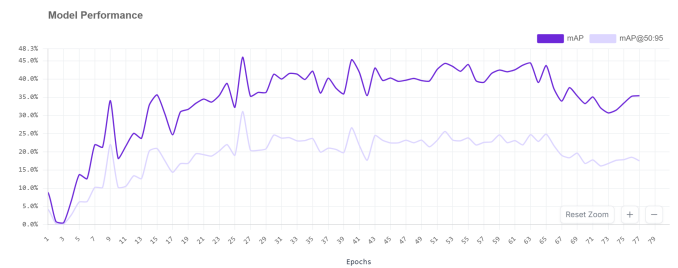


Figure 24. Ver 4 — Model performance (mAP and mAP@50:95) over training epochs.



Figure 25. Ver 4 — Training loss curves (box, class, and overlap components).

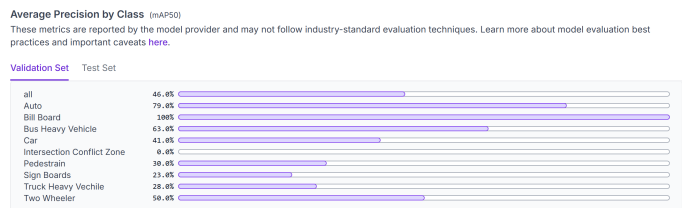


Figure 26. Ver 4 — Model and dataset training configuration details.

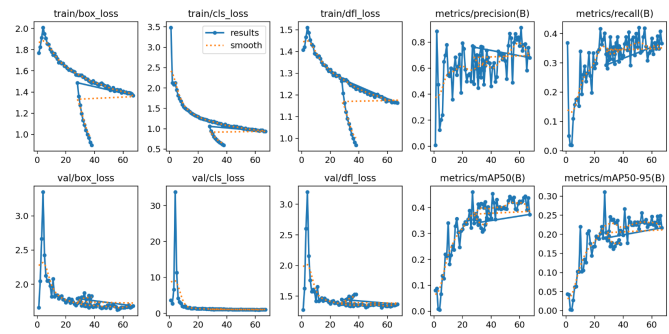


Figure 27. Ver 4 — Advanced training graphs exported from Roboflow.

5. COMPARATIVE RESULTS

Table 2 provides a consolidated comparison of all completed model versions.

Table 2. Validation set performance across all model versions

Ver	Model	mAP@50	Prec.	Rec.
0	RF-DETR Nano	38.0%	33.3%	33.3%
1	YOLOv26 Fast	33.8%	91.6%	17.0%
2	RF-DETR Nano	57.3%	64.3%	43.3%
3	YOLOv11 Fast	46.9%	69.2%	42.5%
4	YOLOv12 Fast	46.0%	77.1%	40.2%

NOTE

All metrics reported here are from the *External Validation Set* as reported by Roboflow. These metrics may not follow industry-standard COCO evaluation protocols. Refer to Roboflow’s evaluation documentation for caveats.

The model comparison reveals several trends:

- **RF-DETR Nano benefits greatly from dataset quality.** Ver 2 (same architecture as Ver 0, cleaner dataset) achieved a 19.3 percentage-point improvement in mAP@50.
- **YOLOv26 Fast is precision-biased.** Ver 1’s 91.6% precision with only 17% recall suggests the model defaults to high-confidence, low-volume predictions.
- **Crop augmentation has a moderate positive effect.** Ver 3 (YOLOv11 + crop) was competitive with the baseline RF-DETR despite being a lighter architecture.
- **Class imbalance persists.** The *Car* class consistently dominates (100% mAP@50 in Ver 0), while *Intersection Conflict Zone* remains very difficult to detect (1% in Ver 0).

6. PROJECT FOLDER STRUCTURE

The project is organized as follows within VS Code:

- .venv/ — Python virtual environment
- extracted_frames/ — Frames sampled at 1 FPS from source video
- report/ — LaTeX report source files
 - main.tex — This document
 - taubabel.sty — Tau multilingual labels package

- `tauenvs.sty` — Tau custom environments package
- `scene_with_gaze.mp4` — Source dashcam video with gaze overlay
- `video_frame_extractor.py` — Frame extraction script
- `requirements.txt` — Python dependencies
- `README.md` — Project overview

7. DISCUSSION

7.1. Dataset Limitations

The primary bottleneck in this study is dataset size. With only 875 base images, the dataset is relatively small for training robust object detectors, particularly for rare classes like *Intersection Conflict Zone*. The crop augmentation in Versions 3 and 4 partially addresses this by increasing scale diversity, but additional real-world data collection would likely yield the greatest improvements.

The class imbalance between *Car* (frequent), *Pedestrian* (moderate), and *Intersection Conflict Zone* (sparse, region-level) presents a fundamental challenge. Future work could apply weighted loss functions or class-balanced sampling to mitigate this.

7.2. Eye Gaze Correlation

The second component of this pipeline—correlating detected objects with eye tracking gaze data—was not evaluated in this report as it constitutes the next stage of the project. Once a sufficiently accurate detector is deployed, the planned approach is:

1. Extract gaze fixation coordinates per frame from the eye tracker output.
2. Run the trained YOLO/RF-DETR model on each frame to obtain bounding box predictions.
3. For each fixation point, check whether it falls within a predicted bounding box.
4. Compute fixation-object co-occurrence statistics to identify which object classes the rider gazed at, for how long, and in which contexts.

This will allow quantitative analysis of rider visual attention across object categories.

7.3. Model Selection for Deployment

Based on current results, **Ver 2 (RF-DETR Nano)** is recommended for the gaze correlation pipeline due to its highest mAP@50. However, if real-time inference speed is required (e.g., for on-device or embedded deployment on the rider's device), **Ver 3 (YOLOv11 Fast)** offers a competitive balance of accuracy and inference speed.

8. CONCLUSION

This report documented a complete pipeline for rider POV object detection across five model versions. Key findings include:

- Frame extraction at 1 FPS and manual Roboflow annotation yielded a 875-image dataset covering three object classes.

- The RF-DETR Nano model (Ver 2) achieved the best validation mAP@50 of 57.3% on the refined dataset.
- Dataset quality improvement had a larger effect than architecture change.
- Crop augmentation and YOLOv11 Fast produced a strong secondary result (Ver 3, 46.9% mAP@50).
- Class imbalance in *Intersection Conflict Zone* remains the primary challenge.

The next phase of the project will integrate gaze tracking data with detected object bounding boxes to analyze rider visual attention patterns at intersections and conflict zones.